SegBlocks: Block-Based Dynamic Resolution Networks for Real-Time Segmentation

Thomas Verelst and Tinne Tuytelaars

Abstract—SegBlocks reduces the computational cost of existing neural networks, by dynamically adjusting the processing resolution of image regions based on their complexity. Our method splits an image into blocks and downsamples blocks of low complexity, reducing the number of operations and memory consumption. A lightweight policy network, selecting the complex regions, is trained using reinforcement learning. In addition, we introduce several modules implemented in CUDA to process images in blocks. Most important, our novel BlockPad module prevents the feature discontinuities at block borders of which existing methods suffer, while keeping memory consumption under control. Our experiments on Cityscapes and Mapillary Vistas semantic segmentation show that dynamically processing images offers a better accuracy versus complexity trade-off compared to static baselines of similar complexity. For instance, our method reduces the number of floating-point operations of SwiftNet-RN18 by 60% and increases the inference speed by 50%, with only 0.3% decrease in mIoU accuracy on Cityscapes.

Index Terms—Conditional execution, convolutional neural networks, model compression, semantic segmentation

1 Introduction

Computational demands of computer vision are constantly increasing, with new deep learning architectures growing in size and new datasets containing images of ever higher resolution. For instance, the Cityscapes dataset for semantic segmentation includes images of 2048 by 1024 pixels [1], and high-resolution images are also used in medical [2] and remote sensing applications [3], [4]. The sheer number of pixels results in a large number of computations due to the convolutions. In addition, many convolutional layers are required to achieve a large receptive field [5].

Deep learning applications using these images are often deployed on low-power devices such as phones, surveillance cameras, or car platforms [6], [7]. The interest in embedded computer vision lead to smaller neural network architectures [8], [9], [10], [11] and model compression methods [12], [13], [14].

These approaches result in static neural networks, which apply identical operations to every pixel. Yet not every image region is equally complex. Therefore, static networks may not be allocating operations in the most optimal way. Various methods have been proposed to dynamically adapt network architectures based on image complexity. Most are designed for classification tasks, for instance by completely skipping network layers or channels [15], [16], [17], [18], [19] and only few methods target dense pixel-wise classification tasks [20], [21], [22]. Dynamic methods typically focus on the theoretical reduction in computational complexity, due to the implementation challenges of sparse operations [18], [19], [23], [24]. In this work, we speed up inference using block-based processing, of which efficient implementations

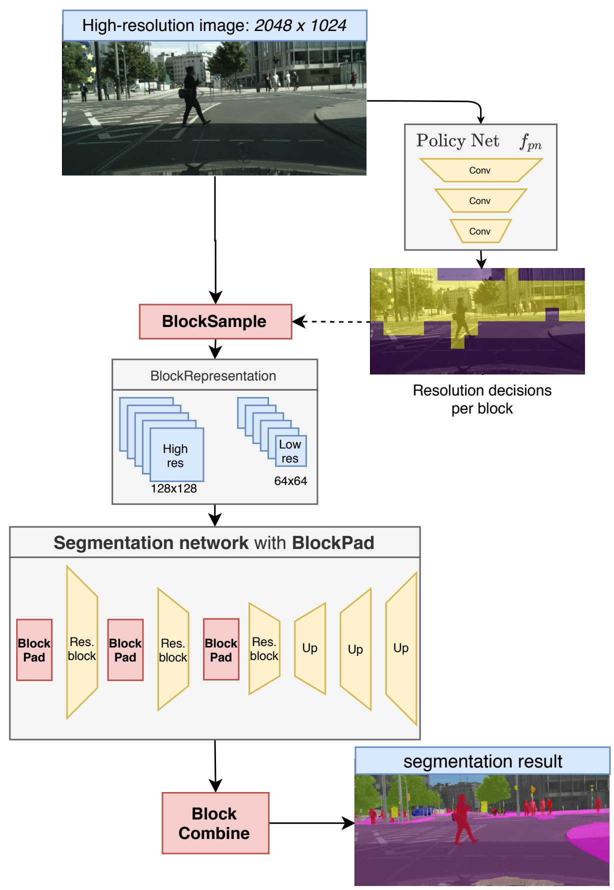
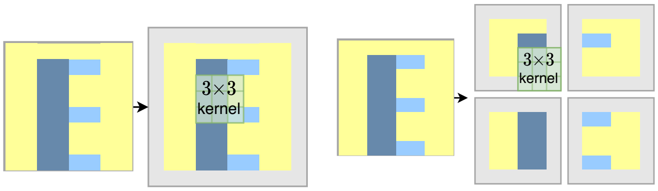


Fig. 1. SegBlocks adjusts the processing resolution of image regions based on their complexity. A lightweight policy network decides which blocks should be processed in high resolution mode. The BlockSample module splits the image and downsamples blocks according to the policy. The resulting BlockRepresentation can be processed by typical deep convolutional networks after replacing zero-padding with our custom BlockPad modules.

• *T. Verelst and T. Tuytelaars are with the Center for Processing Speech and Images, Department Electrical Engineering, KU Leuven. E-mail: {thomas.verelst, tinne.tuytelaars}@esat.kuleuven.be*

Manuscript received October 27, 2020. This work was supported by the FWO-SBO HAPPY-project. This paper was presented in part at the Embedded Vision Workshop at the European Conference on Computer Vision 2020.



(a) Convolution on full image (b) Convolution on blocks

Fig. 2. Illustration of the zero-padding problem when processing blocks: convolutions pad individual blocks with zeros (grey), stopping the propagation of features between blocks and therefore resulting in a loss of global context.

have already been demonstrated on GPU and other platforms [25], [26], [27].

We propose a method to dynamically process low-complexity regions at a lower resolution, as illustrated in Figure 1. Our method splits an image into blocks, and then downsamples non-important blocks. Reducing the processing resolution not only reduces the computational cost, but also decreases the memory consumption. The policy network, determining whether a region should be processed in high or low resolution, is trained using reinforcement learning.

Efficiently processing images in blocks without losing accuracy is not trivial. One could treat each block as a separate image and then combine the individual block outputs. However, features cannot propagate between individual blocks (Figure 2b), leading to a loss in global context and significant decrease in accuracy. Existing works partially address this by including global features extracted by a separate network branch, requiring custom architectures [21], [22].

Our method addresses this problem by replacing zero-padding with our BlockPad operation: this custom CUDA module copies features from neighboring blocks into the padding border and therefore preserves feature propagation between blocks, as if the network was never split into blocks. A comparison between zero-padding and the BlockPad module is given in Figure 3, showing that our module avoids artifacts at block borders.

The contributions of this work are as follows:

- We introduce the concept of dynamic block-based convolutional neural networks, where blocks are downsampled based on their complexity, to reduce their computational cost. In addition, we provide CUDA modules for PyTorch to efficiently implement block-based methods [1].
- We train the policy network with reinforcement learning, in order to select complex regions for high-resolution processing.
- We demonstrate our method using a state of the art semantic segmentation network, and show that our method reduces the number of floating-point operations (FLOPS) and increases the inference speed (FPS) with only a slight decrease in mIoU accuracy. Our method achieves better accuracy than static baseline networks of similar complexity.

1. The code is available at
https://github.com/thomasverelst/segblocks-segmentation-pytorch

2 Related Work

2.1 Semantic segmentation

Traditional works in semantic segmentation focus on improving segmentation accuracy, without taking into account the network complexity [5], [28], [29]. In contrast to other tasks such as classification or object detection, segmentation requires pixel-wise labels having the same resolution as the input. Preserving spatial information in the network requires high-resolution latent representations with a high computational and memory cost. At the same time, a large receptive field is needed to incorporate global context. In order to keep the network size under control, semantic segmentation networks use encoder-decoder architectures with skip connections [29], dilated convolutions [30] and spatial pyramid pooling [5].

Applications such as driverless cars have raised interest in real-time inference on embedded devices. Several high-resolution datasets for these applications have emerged. The Cityscapes dataset [1] provides images of 2048×1024 pixels, with highly detailed annotations for 30 semantic classes. A more extensive dataset is Mapillary Vistas [31] with 25000 high-resolution images and 152 object categories. As real-time inference is crucial for these applications and datasets, smaller and more efficient segmentation networks have been developed for this task specifically.

ICNet [10] proposes a custom encoder architecture processing an image pyramid, with multi-resolution feature maps fused before the decoder. ERFNet [11] factorizes convolution kernels into 1×3 and 3×1 kernels to reduce the computational cost. Bilateral Segmentation Network (BiSeNet) [32] presents a network with two branches: a Spatial Path to encode high-resolution spatial information and a Context Path to achieve a high receptive field. DABNet [33] proposes a Depthwise Asymmetric Bottleneck module to capture local and contextual information. Other methods use attention modules [34], [35], [36], [37] to incorporate global context, for instance by weighting spatial features [38]. SwiftNet [39] demonstrates that state of the art results can be achieved with a more simple architecture: a ResNet-18 encoder [40], pretrained on ImageNet [41], is combined with a spatial pyramid pooling (SPP) module [42] and basic decoder. Model compression techniques can further reduce the computational cost, by applying pruning [12], knowledge distillation [13], quantization [14] or factorization [43], [44].

2.2 Dynamic segmentation

These efficient architectures and compression techniques achieve impressive performance on segmentation tasks, but do not exploit spatial properties of the image as they process each pixel with the same operation. Some recent methods address this by processing the image dynamically in blocks or patches. It is worth noting that dynamic methods are often complementary to static compression methods, and both can be combined to further reduce the computational cost.

The method proposed by Huang *et al.* [20] combines a fast segmentation model with a large model. First, the image is processed by a fast network, such as ICNet [10]. Then, a selection criterion, based on the softmax certainty

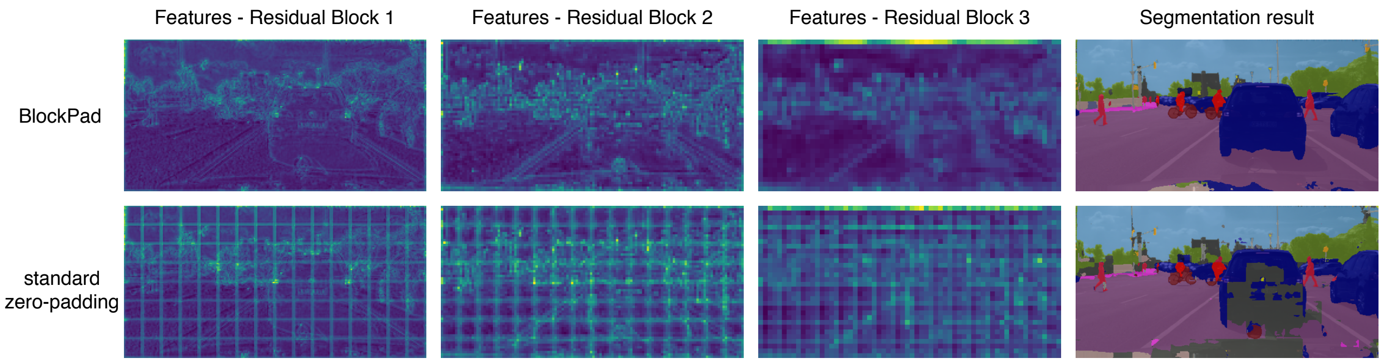


Fig. 3. Comparison of feature maps when processing an image in blocks with either standard zero-padding or our custom BlockPad module. Features are visualized by summing activation magnitudes over the channel dimension and combining the outputs of individual blocks into a single image. Standard zero-padding introduces noticeable artifacts at block borders. In addition, the output shows inconsistencies, e.g. in the car, due to the lack of global context for individual blocks. In contrast, our BlockPad module enables feature propagation is if the network was never evaluated in blocks.

of the output, uses the rough predictions to decide which image regions should be re-evaluated by the large model (e.g. PSPNet [28]). Such a two-stage approach has two drawbacks. First, the large network processes the image in blocks and the discontinuities at block borders stop feature propagation. Therefore, the method uses large patches of at least 256×256 pixels to include some global context. Secondly, the feature maps and predictions of the small network are not re-used by the large network, making the method less efficient and only applicable when the accuracy and cost difference between the small and large network is substantial.

Wu *et al.* [22] propose a custom architecture where a high-resolution branch improves rough predictions of a low-resolution branch. Again, this method struggles with the feature propagation problem at patch borders and therefore uses blocks of 256×256 pixels. In addition, they incorporate global features from the low-resolution network into the high-resolution branch to provide more global context. The Patch Proposal Network [21] method of Wu *et al.* has a similar architecture, with a low-resolution global branch and high-resolution refinement branch. The local refinement branch only operates on selected patches, based on a trained selection criterion where regions with below-average accuracy are refined. Features of both branches are fused before generating final predictions.

In contrast to these methods, our dynamic method does not require modifications to existing network architectures. We address the discontinuities at block borders without requiring additional global features in the patch-based processing. Furthermore, by being applicable on existing architectures, our method benefits from further advancements in network architectures.

Another approach to reduce spatial redundancy is to warp the input image to enlarge important regions [45]. The work of Marin *et al.* [46] uses non-uniform image sampling to focus on complex regions. However, as the internal representation of the network is still a regular-spaced pixel grid, the flexibility of non-uniform downsampling is limited, and the network typically focuses on a single region.

2.3 Conditional execution

Adapting the network architecture based on the input image is also known as *conditional execution* [47], [48], [49] or dynamic neural networks. For instance, some methods reduce the processing cost of simple images by skipping complete residual blocks [15], [16], [17] or by dynamically pruning feature channels [18]. However, these methods are intended for classification tasks, with a large variety in features between images of different classes. In segmentation, many images contain the same objects and features in different spatial arrangements.

Some dynamic methods rely on spatial properties to reduce the computational cost: they skip computations on low-complexity regions. Spatially Adaptive Computation Time [24] and cascade-based methods [23] halt the computation of features when features are 'good enough'. DynConv [19] demonstrates inference speed improvements on human pose estimation tasks, where large regions of the image can be completely ignored by the network. The methods of Xie *et al.* [50] and Figurnov *et al.* [51] dynamically interpolate features in low-complexity regions. Requiring sparse convolutions, these methods either demonstrate no inference speed improvements [24], only on depthwise convolutions [19] or only on CPU [24], [50], or are intended for specialized hardware [52]. Block-based approaches are considered to be more feasible to implement efficiently on most platforms [25], [26], [27]. In addition, methods completely skipping non-complex regions are less suitable for pixel-wise labeling tasks such as segmentation. Our method processes every image region, but reduces the cost of non-complex regions.

An important aspect of conditional execution is the policy which determines the layers, channels or regions to execute. Often, this policy cannot be trained by standard backpropagation due to its discrete output. Recently, the Gumbel-Softmax [53], [54] trick gained popularity in dynamic methods [16], [18], [19]. In our case, it would require predictions for each region in both high and low-resolution, increasing the computational cost at training time. Instead, we show that the policy can be trained efficiently using reinforcement learning [55].

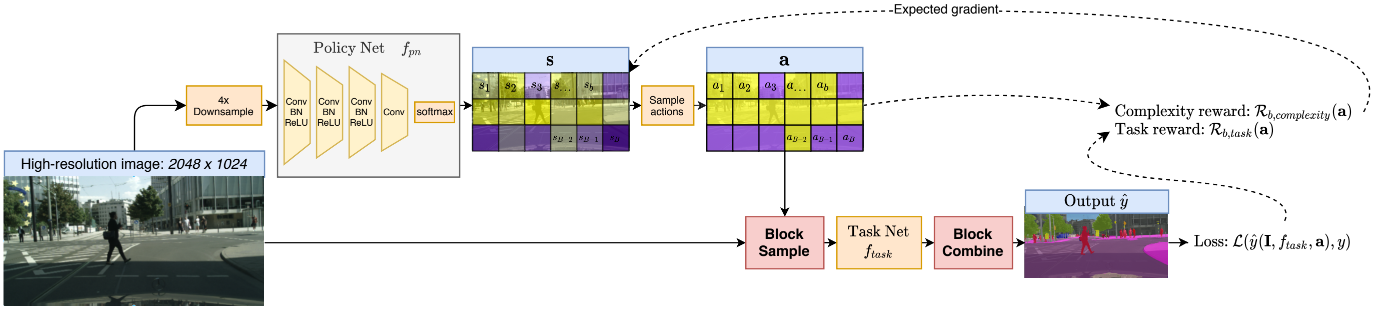


Fig. 4. Training the policy network with reinforcement learning: The lightweight policy net predicts soft decisions $\boldsymbol{s}$, which are sampled to actions $\boldsymbol{a}$. The policy net's gradients are estimated using REINFORCE [56] with a separate reward per block, based on the task loss in a block's region and the number of blocks processed at high resolution.

3 Method

SegBlocks splits an image into blocks and adjusts the processing resolution of each block based on its complexity. A lightweight policy network processes a low-resolution version of the image and outputs resolution decisions for each image region. For implementation simplicity, we restrict the resolution to two options, with either high- or low-resolution processing. Note that high- and low-resolution regions are processed by the same convolutional filters, which consequently perceive objects at different scales. This does not affect the performance negatively though, as segmentation images typically have large variations in object size, and convolutional neural networks are trained to be robust against scale variations. Since resolution decisions are discrete, the policy network is trained with reinforcement learning, using a reward function to determine the expected gradient.

First, we discuss how the reinforcement learning is incorporated using a reward taking into account both computational complexity and task accuracy. The policy is trained jointly with the main segmentation network. In the second part, we elaborate on custom modules making block-based adaptive processing of images possible.

3.1 Downsampling policy

3.1.1 Policy network

The policy network is a convolutional neural network that outputs a discrete decision per block. The network should be small compared to the main segmentation network. We use a simple architecture with 4 convolutional layers of 64 channels, batch normalization and ReLU non-linearities followed by a softmax layer over the channel dimension. A four times downsampled version of the image is given as input. The policy network f_{pn}, parametrized by θ, outputs probabilities s_b per block, indicating the likelihood that the block should be processed in high resolution based on the input image $\boldsymbol{I}$:

$$\boldsymbol{s} = f_{pn}(\boldsymbol{I}; \theta), \tag{1}$$

$$\text{with } \boldsymbol{s} = [s_1, \ldots, s_b, \ldots, s_B] \in [0, 1]^B. \tag{2}$$

The soft probabilities are sampled to actions $\boldsymbol{a}$, where $a_b = 1$ results in high-resolution processing of block b:

$$s_b = P(a_b = 1). \tag{3}$$

Standard backpropagation cannot be applied due to the sampling of discrete actions. Finding optimal decisions $\boldsymbol{a}$ for context $\boldsymbol{I}$, with a constrained budget, is also referred to in literature as contextual bandits [55]. This can be seen as a reinforcement learning scenario with a single time step: the policy network predicts all actions a_b at once and directly obtains the reward. The probability of actions $\boldsymbol{a}$, for image $\boldsymbol{I}$ and parameters θ is given by

$$\pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) = \prod_{b=1}^{B} p_\theta(a_b \mid \boldsymbol{I}). \tag{4}$$

The objective is to find policy parameters θ, so that the predicted actions $\boldsymbol{a}$ for image $\boldsymbol{I}$ maximize the policy reward.

3.1.2 Policy reward

The goal of the policy network is to select the most important blocks for high-resolution processing. To avoid early convergence to a sub-optimal state where all blocks are processed at high resolution, the reward takes into account both the number of blocks processed at high resolution and the task accuracy. Figure 4 illustrates how both rewards are integrated in the training process.

We assume that a resolution decision a_b only affects the segmentation output of block b and does not influence adjacent blocks. The total reward for an image can then be expressed as the sum of individual block rewards. A block reward consists of one term optimizing accuracy and second term keeping the number of operations under control, weighted by hyperparameter γ. The reward per image is then written as

$$\begin{aligned} \mathcal{R}(\boldsymbol{a}) &= \sum_{b}^{B} \mathcal{R}_b(\boldsymbol{a}) \\ &= \sum_{b}^{B} \left(\mathcal{R}_{b,task}(\boldsymbol{a}) + \gamma \mathcal{R}_{b,complexity}(\boldsymbol{a})\right). \end{aligned} \tag{5}$$

The percentage of blocks processed in high-resolution is given by

$$\sigma(\boldsymbol{a}) = \frac{1}{B} \sum_{b=1}^{B} a_b. \tag{6}$$

Then, the following reward minimizes the difference between σ and the desired percentage τ:

$$\mathcal{R}_{b,complexity}(\boldsymbol{a}) = \begin{cases} -(\sigma(\boldsymbol{a}) - \tau) & \text{if } a_b = 1, \\ \sigma(\boldsymbol{a}) - \tau & \text{if } a_b = 0. \end{cases} \tag{7}$$

This reward is positive for blocks processed at high resolution when the actual percentage σ is smaller than the desired percentage τ, resulting in an incentive for the policy network to process more blocks at high resolution. Using a target τ instead of simply minimizing σ results in more stable training and lower sensitivity to hyperparameter γ.

The task reward encourages the policy network to select regions with a high segmentation loss for high resolution processing. In general, those regions correspond to the most complex ones in the image. The task criterion, e.g. pixel-wise cross entropy, is denoted by $\mathcal{L}$. The task loss per image is then given by

$$L_{task} = \mathcal{L}(\widehat{y}, y) \tag{8}$$

where $\widehat{y}$ and y denote the predictions and ground truth labels respectively. Our reward is based on the task loss per block. Values $\widehat{y}_b$ and y_b are obtained by only considering values in the block region. The task reward of block b can then be defined as

$$\mathcal{R}_{b,task}(\boldsymbol{a}) = \begin{cases} \mathcal{L}(\widehat{y}_b, y_b) - L_{task} & \text{if } a_b = 1, \\ -(\mathcal{L}(\widehat{y}_b, y_b) - L_{task}) & \text{if } a_b = 0. \end{cases} \tag{9}$$

Subtracting L_{task} is not strictly necessary but reduces the variance of the rewards.

3.1.3 *Expected gradient*

The policy network should predict actions that maximize the expected reward:

$$\max \mathcal{J}(\theta) = \max \mathbb{E}_{\boldsymbol{a} \sim \pi_\theta}[\mathcal{R}(\boldsymbol{a})]. \tag{10}$$

The policy network's parameters θ can then be updated using gradient ascent with learning rate α:

$$\theta \leftarrow \theta + \alpha \nabla_\theta [\mathcal{J}(\theta)] \tag{11}$$

The gradients of J can be derived similarly to the policy gradient method REINFORCE [56]:

$$\begin{aligned} \nabla_\theta \mathcal{J}(\theta) &= \nabla_\theta \mathbb{E}_{\boldsymbol{a}}[\mathcal{R}(\boldsymbol{a})] \\ &= \nabla_\theta \sum_{\boldsymbol{a}} \pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) \mathcal{R}(\boldsymbol{a}) \\ &= \sum_{\boldsymbol{a}} \nabla_\theta \pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) \mathcal{R}(\boldsymbol{a}) \\ &= \sum_{\boldsymbol{a}} \pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) \nabla_\theta [\log \pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) \mathcal{R}(\boldsymbol{a})] \\ &= \sum_{\boldsymbol{a}} \pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) \nabla_\theta [\log \prod_{b=1}^{B} p_\theta(a_b \mid \boldsymbol{I})] \mathcal{R}(\boldsymbol{a}) \\ &= \sum_{\boldsymbol{a}} \pi_\theta(\boldsymbol{a} \mid \boldsymbol{I}) \mathcal{R}(\boldsymbol{a}) \sum_{b=1}^{B} \big(\nabla_\theta \log p_\theta(a_b \mid \boldsymbol{I})\big) \\ &= \mathbb{E}_{\boldsymbol{a}} \Big[\mathcal{R}(\boldsymbol{a}) \sum_{b=1}^{B} \big(\nabla_\theta \log p_\theta(a_b \mid \boldsymbol{I})\big)\Big] \\ &= \mathbb{E}_{\boldsymbol{a}} \Big[\sum_{b=1}^{B} \big(\mathcal{R}_b(\boldsymbol{a}) \nabla_\theta \log p_\theta(a_b \mid \boldsymbol{I})\big)\Big]. \end{aligned} \tag{12}$$

In practice, the expectation of Equation 12 is approximated with Monte-Carlo sampling using samples in the mini-batch. The result can be interpreted as applying REINFORCE on each block individually with reward $\mathcal{R}_b(\boldsymbol{a})$, giving unbiased but high-variance gradient estimates. In practice, we found the policy network stable to train for a large range of hyperparameters.

The segmentation network and policy network are jointly trained. The hybrid loss to be minimized is then given by

$$L_{hybrid} = L_{task} + \frac{\beta}{N} \sum_{n=1}^{N} \sum_{b=1}^{B} \big(\mathcal{R}_b(\boldsymbol{a}) \log p_\theta(a_b \mid \boldsymbol{I})\big) \tag{13}$$

with N the batch size.

3.2 Block Modules

We introduce three new modules for block-based processing with convolutional neural networks: BlockSample, BlockPad, and BlockCombine. Figure 5 gives an overview of a simple block-processing pipeline.

3.2.1 *BlockSample*

The BlockSample module splits the image into blocks and downsamples low-complexity blocks consecutively. Average pooling is used for efficient downsampling. We use a downsampling factor of 2, reducing the computational cost of low-complexity regions by a factor 4. The CUDA implementation fuses the splitting and downsampling steps into a single operation. Images are split into blocks of a predefined block size, for instance 128×128 pixels. Downsampling operations, such as max pooling, further reduce the spatial dimensions of blocks throughout the network.

3.2.2 *BlockPad*

The BlockPad module replaces zero-padding and eliminates discontinuities at block borders by copying features from neighboring blocks into the padding. Evaluating the network with only high-resolution blocks is equivalent to normal processing without blocks.

When two high-resolution or two low-resolution blocks are adjacent, pixels can be copied directly while preserving their spatial relationship. When copying features from low-resolution blocks into the padding of high-resolution blocks, features are nearest neighbor upsampled in order to preserve the spatial relationship, as illustrated in Figure 5. When copying features from high-resolution blocks into the low-resolution block's padding, we considered two possibilities: strided subsampling and average sampling. Figure 6a illustrates that subsampling copies features in a strided pattern., whilst average sampling (Fig 6b) combines multiple pixels. Our experiments show that average sampling achieves better performance with a small increase in overhead.

3.2.3 *BlockCombine*

The BlockCombine module upsamples low-resolution blocks to the same resolution as the high-resolution ones, and then combines all blocks into a single tensor. Our CUDA implementation merges the nearest neighbour upsampling and block combining steps to reduce the overhead of this operation.

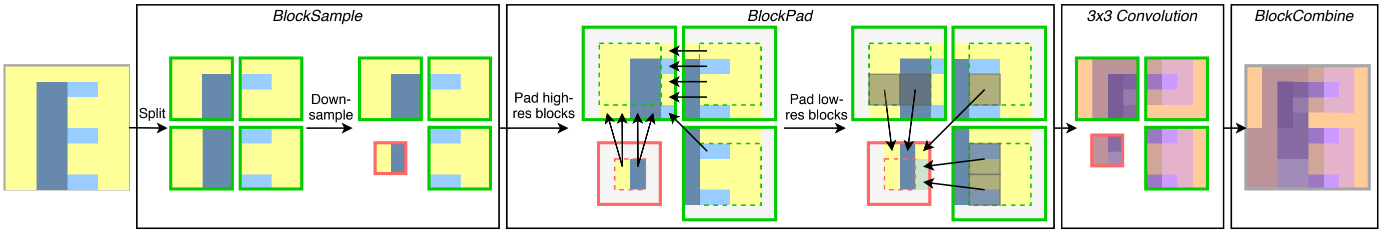
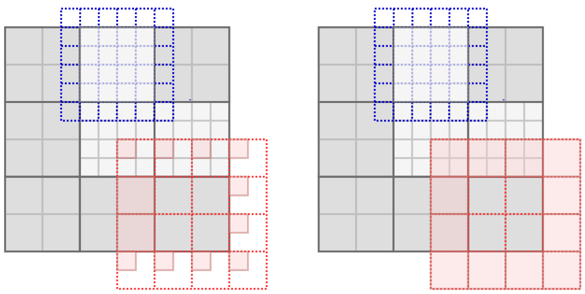


Fig. 5. Illustration of the BlockSample, BlockPad and BlockCombine modules. BlockSample splits the image and downsamples low-complexity blocks. BlockPad replaces zero-padding and enables feature propagation between blocks. The module copies features from neighboring blocks into the padding. When padding low-resolution blocks adjacent to high-resolution blocks, multiple pixel values of the high-resolution blocks are averaged to better preserve the spatial coherence of features.



(a) Strided subsampling (b) Average sampling

Fig. 6. BlockPad aims to respect the spatial relationship between high- and low-resolution blocks by sampling using the illustrated patterns. Low-resolution blocks of resolution 2×2 pixels are colored dark grey, while the 4×4 high-resolution blocks are colored light grey. The blue and red grid shows the sampling pattern when padding high-resolution and low-resolution blocks respectively.

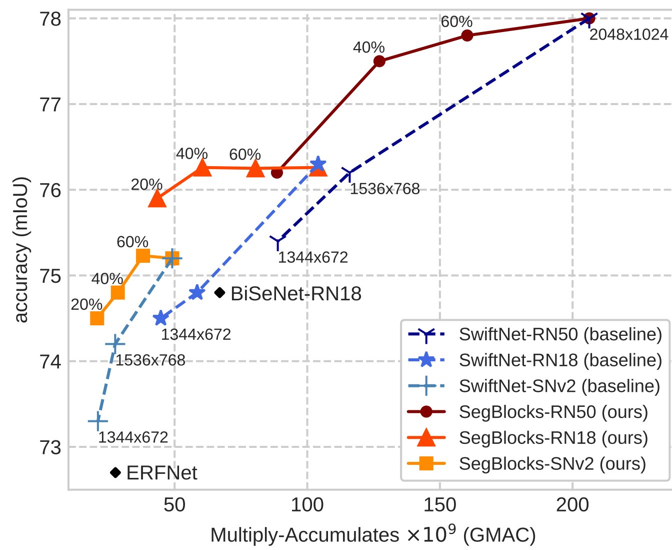


Fig. 7. Cityscapes validation results and comparison with static baselines of similar complexity. For a given computational complexity (GMACs), our adaptive resolution method SegBlocks consistently outperforms static networks trained at lower resolution, as well as smaller static backbones. Each backbone is trained and evaluated with $\tau \in 0.2, 0.4, 0.6$ for dynamic models and resolutions 2048×1024, 1536×768, 1344×672 for static baselines.

4 Experiments

We integrate our dynamic processing method in SwiftNet [39], a state of the art network for real-time semantic segmentation of road scenes. We test semantic segmentation on the Cityscapes [1] and Mapillary Vistas [31] datasets. In addition, we provide an ablation study, analyzing the overhead of block-based processing. Our method is implemented in PyTorch 1.5 and CUDA 10.2, without TensorRT optimizations. We report two model complexity metrics: the number of computations and the frames per second.

The number of computations is reported in billions of *multiply-accumulates (GMACs)* per image, being half the number of floating-point operations (FLOPS). The FLOPS metric is often used to compare model complexity in a platform-agnostic manner, since the efficiency of the implementation has no impact. For our dynamic method, where the number of computations varies per image, we report the average GMAC count.

Frames per second (FPS) is a more practical metric to measure inference speed, but depends largely on the implementation of building blocks. Therefore, both FPS and GMACs metrics should be taken into account for fair comparison. We measure the inference speed on both a high-end Nvidia GTX 1080 Ti 11GB GPU and low-end Nvidia GTX 1050 Ti 4 GB GPU, paired with an Intel i7 processor. The model is warmed up on the first half of the image set, and the speed is measured on the second half. To compare the inference speed to those reported by others, we normalize the FPS numbers (*norm FPS*) of different GPUs based on their relative performance, using the same scaling factors as Orsic *et al.* [39].

4.1 Cityscapes semantic segmentation

The Cityscapes [1] dataset for semantic segmentation consists of 2975 training, 500 validation and 1525 test images of 2048×1024 pixels. We use the standard 19 classes for semantic segmentation and do not use of the extra coarsely labeled images.

We train SwiftNet [39] with three different backbones: ResNet-18 (RN18), ResNet-50 (RN50) [40], and ShuffleNet-V2 [9]. Our dynamic processing method is integrated in these baseline networks and we trained models for different $\tau \in [0, 1]$, indicating the desired percentage of high-resolution blocks. The models are trained on 768×768 crops, augmented by image scaling between factor 0.5 and 2, $[-10, 10]$ degrees of rotation, color jitter and random horizontal flip. The optimizer is Adam, the learning rate is cosine annealed from $4e{-}4$ to $1e{-}6$ over 300 epochs, weight decay is set to $1e{-}4$ and the batch size is 8. The backbone is

TABLE 1
Results on Cityscapes semantic segmentation. Our SegBlocks models are based on the respective SwiftNet baselines, and integrate block-based dynamic resolution processing in these networks.

	Method	mIoU *val*	*test*	GMACs	FPS	norm FPS	memory (max)
Our method	SwiftNet-RN50 (baseline, our impl.)	78.0	-	206.3	16.3 @ 1080 Ti, 3.8 @ 1050 Ti	16.3	2724 MB
	SegBlocks-RN50 (τ=0.4)	77.5	-	127.2	23.3 @ 1080 Ti, 5.7 @ 1050 Ti	23.3	2957 MB
	SegBlocks-RN50 (τ=0.2)	76.2	-	88.5	30.0 @ 1080 Ti, 7.3 @ 1050 Ti	30.0	2044 MB
	SwiftNet-RN18 (baseline, our impl.)	76.2	74.4	104.1	38.4 @ 1080 Ti, 9.9 @ 1050 Ti	38.4	2256 MB
	SegBlocks-RN18 (τ=0.4)	76.3	73.8	60.5	48.6 @ 1080 Ti, 13.5 @ 1050 Ti	48.6	1862 MB
	SegBlocks-RN18 (τ=0.2)	75.9	-	43.5	57.8 @ 1080 Ti, 16.8 @ 1050 Ti	57.8	1928 MB
	SwiftNet-SNv2 (baseline, our impl.)	75.2	-	49.1	34.7 @ 1080 Ti, 10.5 @ 1050 Ti	34.7	1338 MB
	SegBlocks-SNv2 (τ=0.4)	74.8	73.8	27.6	45.3 @ 1080 Ti, 13.1 @ 1050 Ti	45.3	1210 MB
	SegBlocks-SNv2 (τ=0.2)	74.5	-	20.6	56.0 @ 1080 Ti, 16.1 @ 1050 Ti	56.0	1192 MB
Dynamic networks	Patch Proposal Network (AAAI2020) [21]	75.2	-	-	24 @ 1080 Ti	24	1137 MB
	Huang et al. (MVA2019) [20]	76.4	-	-	1.8 @ 1080 Ti	1.8	-
	Wu et al. [22]	72.9	-	-	15 @ 980 Ti	33	-
	Learning Downsampling (ICCV2019) [46]	65.0	-	34	-	-	-
Efficient static networks	ShelfNet18-lw (CVPR2019) [57]	-	74.8	95	36.9 @ 1080 Ti	36.9	-
	SwiftNet-RN18 (CVPR2019) [39]	75.4	-	104.0	39.9 @ 1080 Ti	39.9	-
	DABNet (BMVC2019) [33]	70.1	-	-	27.7 @ 1080 Ti	27.7	-
	BiSeNet-RN18 (ECCV2018) [32]	74.8	74.7	67	65.5 @ Titan Xp	58.5	-
	ICNet (ECCV2018) [10]	-	69.5	30	30.3 @ Titan X	49.7	-
	GUNet (BMVC2018) [58]	-	70.4	-	33.3 @ Titan Xp	29.7	-
	ERFNet (TITS2017) [11]	72.7	69.7	27.7	11.2 @ Titan X	18.4	-

TABLE 2
IoU per class on the Cityscapes validation set: our method improves the IoU score for most classes. Instance classes such as person, rider, car, bus and train benefit more from high-resolution processing. The percentage of pixels processed in high-resolution, given per class, shows that our method mostly processes road and sky regions at low resolution.

	road	side-walk	buil-ding	wall	fence	pole	traffic light	traffic sign	vege-tation	terrain	sky	per-son	rider	car	truck	bus	train	motor-cycle	bicycle	mIoU
SwiftNet-RN18	97.8	82.9	91.8	50.9	56.2	63.0	69.0	78.1	92.2	61.4	94.7	80.5	60.0	94.8	78.3	84.3	75.0	6.5	76.3	76.2
SwiftNet-RN18 (1536×768)	**97.8**	**83.0**	91.5	54.1	52.8	60.5	65.5	**76.4**	91.9	**62.1**	94.6	79.2	58.2	94.2	75.5	80.5	69.6	57.6	74.9	74.7
SegBlocks-RN18 (τ=0.4)	**97.8**	82.9	**91.9**	**59.5**	**57.6**	**61.1**	**67.0**	76.2	**92.1**	61.9	**94.4**	**80.4**	**59.9**	**94.4**	**77.2**	**82.4**	**77.2**	**60.1**	**75.7**	**76.3**
% high-res pixels	*8.5*	*45.8*	*64.0*	*67.0*	*88.5*	*86.3*	*85.6*	*82.2*	*59.7*	*68.2*	*36.7*	*93.3*	*96.2*	*66.2*	*61.7*	*68.9*	*85.0*	*90.9*	*93.2*	-

pre-trained on ImageNet [41]. Our method uses a block size of 128×128 pixels, resulting in 16×8 blocks per image that can adapt the processing resolution to the content.

Figure 7 shows the mIoU accuracy of models at various computational costs (GMACs). Static baseline networks of different computational complexity are obtained by adjusting the training and evaluation resolution to 2048×1024, 1536×768 and 1344×672. The complexity of our SegBlocks methods is determined by the number of high-resolution blocks, changed by training for different values of hyperparameter $\tau \in \{0.2, 0.4, 0.6\}$. Our SegBlocks methods consistently outperform the static baseline networks, showing that dynamic resolution processing is more beneficial than sampling all regions at lower resolution. For instance, SegBlocks-RN18 with $\tau = 0.4$ reduces the computational complexity of the baseline network by 40%, without decreasing the mIoU. In contrast, a static baseline network of similar complexity, trained on images of 1536×768 pixels, has 1.7% lower mIoU.

Table 1 compares the performance of our method with other dynamic methods and non-dynamic efficient segmentation architectures. Our method outperforms other dynamic methods, such as Patch Proposal Network [21] or the work by Huang et al. [20], by achieving better accuracy at faster inference speeds (FPS) and fewer operations (GMACs). We are also competitive with state of the art architectures for fast semantic segmentation, such as BiSeNet [32] or ERFNet [11]. It is worthwhile to note that our proposed dynamic resolution method is complementary to further improvements in network architectures.

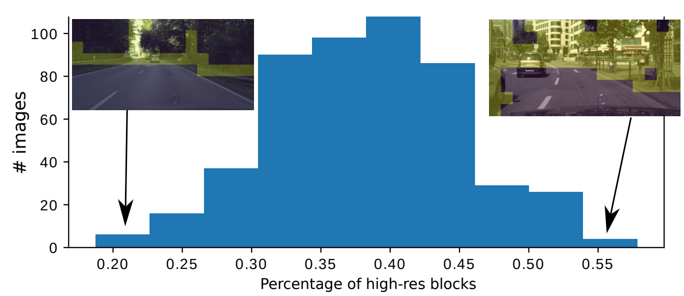


Fig. 8. Percentage of high-resolution blocks per image, as a histogram over the 500 validation images of Cityscapes, for SegBlocks-RN18 (τ=0.4). Simple images use fewer high-resolution blocks, down to 20%, and complex images process up to 55% of the blocks in high resolution (colored yellow).

The table also demonstrates the inference speed improvement achieved using our efficient implementation.

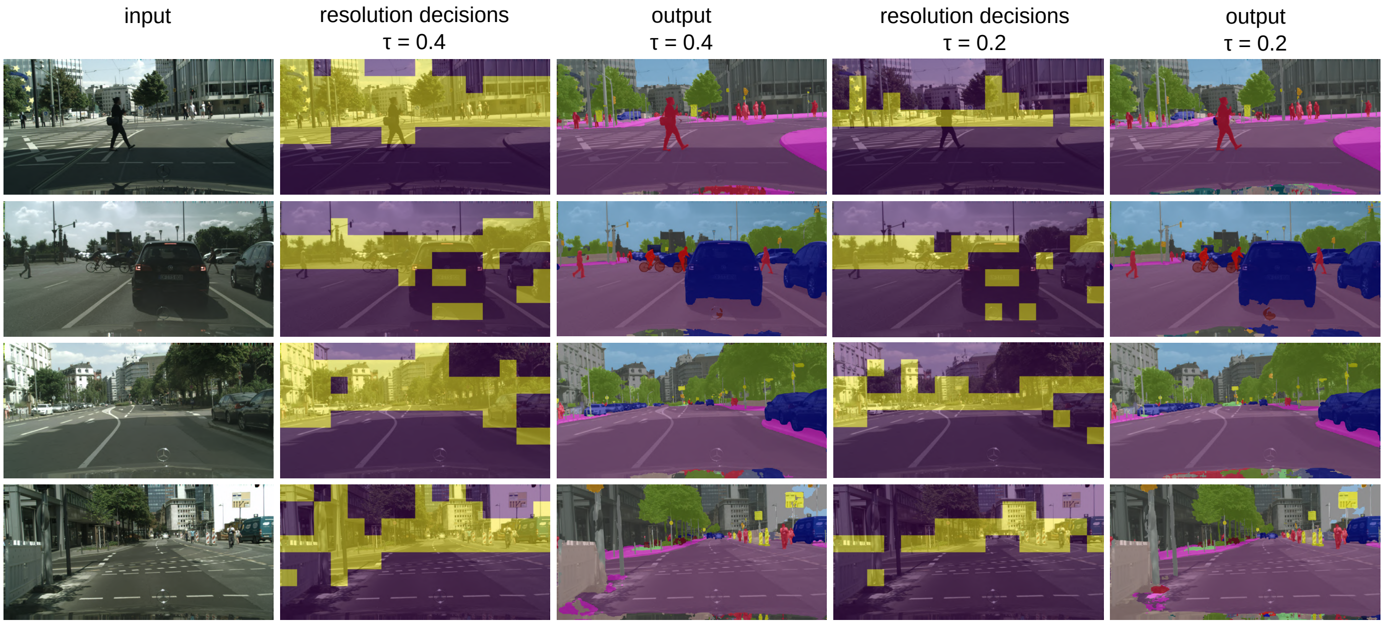


Fig. 9. Examples of resolution decisions made by the policy network and corresponding segmentation outputs, for SegBlocks-RN18 with $\tau = 0.4$ and $\tau = 0.2$. High-resolution blocks are colored in yellow.

For instance, our method improves the inference speed of SwiftNet-RN50 from 16 FPS to 30 FPS on a GTX 1080 Ti, achieving real-time performance. This increase of 84 percent is obtained by reducing the theoretical complexity (GMACs) with 63%, while the mean IoU is decreased by 1.8%. Finally, the memory usage, being the maximum memory consumption measured during inference over the validation set, indicates that our method also reduces memory consumption by storing low-complexity regions at lower resolution.

The IoU result per class is provided in Table 2, combined with the percentage of pixels processed in high-resolution for each class. Our method mainly processes classes such as road, sky and vegetation in low resolution, whereas rider, motorcycle and bicycle are typically sampled in high resolution. Our method adapts the operations to each individual image, and Figure 8 shows the distribution of operations per image. Qualitative results are shown in Figure 9, visualizing the resolution decisions of the policy network and the respective segmentation output, for τ set to 0.4 and 0.2. The policy network, trained with reinforcement learning, properly selects complex regions for high-resolution processing.

4.2 Mapillary Vistas semantic segmentation

Mapillary Vistas [31] is a large dataset containing high-resolution road scene images with labels for semantic and instance segmentation. We use the standard 15000/2000 train/val split, resize all images to 2048×1024 pixels and use the 66 classes for semantic segmentation. We report validation results, since the test server is not available. The hyperparameters are the same as the Cityscapes experiments, but the model is trained for 150 epochs without rotation and color jitter augmentations due to the larger dataset. Similar to the experiments on Cityscapes, we integrate our method in the SwiftNet-RN50 baseline network.

Table 3 demonstrates that SegBlocks also outperforms static baseline networks on this dataset, resulting in better accuracy when comparing models with either similar computational complexity (GMACs) or inference speed (FPS).

TABLE 3
Mapillary Vistas validation results for semantic segmentation. The SwiftNet-RN50 [39] baseline is trained and evaluated at various resolutions. Our dynamic SegBlocks models outperform static baselines of similar complexity. The FPS is measured on an Nvidia GTX 1080 Ti.

Method	mIoU	GMACs	FPS
SwiftNet-RN50 (2048×1024) (our impl.)	39.8	207.1	15.2
SwiftNet-RN50 (1536×768) (our impl.)	38.3	116.5	28.7
SwiftNet-RN50 (1344×672) (our impl.)	38.0	89.1	37.0
SegBlocks-RN50 (τ=0.4)	39.7	113.5	21.5
SegBlocks-RN50 (τ=0.2)	39.4	84.4	28.3

4.3 Ablation

4.3.1 *Module execution time analysis*

We profile the time characteristics of our block modules to analyze their overhead. Table 4 compares the execution time of the Policy Net, BlockSample, BlockPad and BlockCombine modules with the total runtime. The overhead of the Policy Net, BlockSample and BlockCombine modules is negligible, and the overhead of the BlockPad operation is reasonable with around 10 percent of the total execution time. The BlockPad module has less impact on the ResNet-50 backbone, as its 1×1 convolutions in the bottleneck function do not require padding. Note that the cost of the BlockPad operation scales with the total number of processed pixels, and thus with the number of high-resolution blocks.

4.3.2 *Block size*

Table 5 compares the impact of the method's block sizes on accuracy and performance. Figure 10 demonstrates the

TABLE 4
Time profiling of block modules, as total time taken during the inference of 250 validation images on an Nvidia GTX 1080 Ti GPU.

Method	mIoU	GMACs	policy GMACs	Runtime	Policy Net	BlockSample	BlockPad	BlockCombine
SegBlocks-RN50 (τ=0.4)	77.5%	127.2	0.09	11.6 s	0.04 s (<1%)	0.12 s (1%)	0.75 s (6%)	0.04 s (<1%)
SegBlocks-RN50 (τ=0.2)	76.2%	88.6	0.09	9.3 s	0.04 s (<1%)	0.13 s (1%)	0.61 s (7%)	0.03 s (<1%)
SegBlocks-RN18 (τ=0.4)	76.3%	60.5	0.09	5.8 s	0.04 s (<1%)	0.11 s (2%)	0.68 s (12%)	0.02 s (<1%)
SegBlocks-RN18 (τ=0.2)	75.9%	43.5	0.09	4.9 s	0.04 s (<1%)	0.12 s (2%)	0.56 s (11%)	0.02 s (<1%)

TABLE 5
Block size comparison for SegBlocks-RN18 with $\tau=0.4$ on the Cityscapes validation set

Block size	mIoU	GMACs	FPS (1050 Ti)
64×64	76.3%	59.3	11.3
128×128	76.3%	60.5	13.5
256×256	75.2%	56.4	14.6

Fig. 10. Granularity of blocks of 64, 128 and 256 pixels on Cityscapes' images of 2048×1024 pixels.

TABLE 6
Comparison of BlockPad sampling patterns and zero-padding for SegBlocks-RN18 ($\tau=0.4$)

Method	mIoU	Runtime	BlockPad time
Avg-sampling	76.3%	5.8 s	0.68 s (12%)
Strided sampling	75.6%	5.7 s	0.63 s (11%)
Zero-padding	70.2%	5.2 s	N.A. (integrated in conv.)

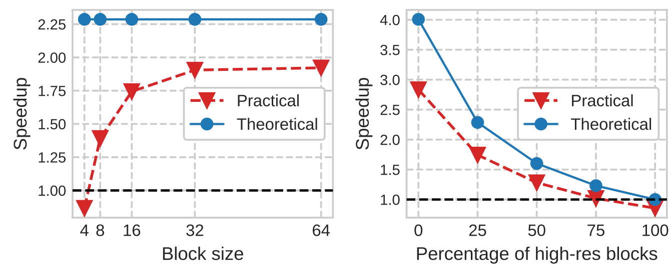
granularity of each block size on Cityscapes. Larger block sizes offer better inference speeds, by padding fewer pixels and having less overhead, whereas smaller block sizes offer finer control of adaptive processing.

4.3.3 *Sampling pattern*

As described in Section 3.2.2, the BlockPad module pads individual blocks by sampling their neighbors. When low-resolution blocks are padded with features of high-resolution blocks, these features should be subsampled. Table 6 compares average-sampling with strided subsampling and shows that average sampling achieves better accuracy (mIoU) with only a slight increase in overhead. In addition, we show that using zero-padding has a significant impact on accuracy, highlighting the importance of BlockPad when processing images in blocks.

4.3.4 *Single residual block analysis*

The overhead introduced by our block modules strongly depends on the block size: larger blocks have fewer pixels in the padding, and require fewer copy operations. Moreover, standard operations, implemented in PyTorch and cuDNN, are often optimized for larger spatial dimensions. We analyze a single residual block to measure the impact of both block size and the percentage of high-resolution blocks.



(a) Block size impact (b) Percentage impact

Fig. 11. Comparison of theoretical versus practical speedup for a single residual block of ResNet-18, studying the impact of the block size and the percentage of high-resolution blocks on the performance.

Figure 11a studies the impact of the block size, when evaluating half of the blocks at high and half of the blocks at low resolution. The number of floating-point operations is reduced by 56%, resulting in a theoretical 2.3 times speed increase. In practice, we measure 1.92 times faster inference of the residual block when the block size is larger than 32. Block sizes smaller than 4 pixels result in slower inference. Note that, even though block sizes are typically large at the network input, e.g. 128×128, downsampling in the network reduces the block size by the same factor. The SwiftNet network downsamples by a factor 32 throughout the network, resulting in block sizes of 4×4 for the deepest network layers. Figure 11a shows that, when evaluating using block size 8, the percentage of high-res blocks should be lower than 75% to achieve practical speedup. For lower percentages, the practical speedup is around 70 percent of the theoretical one.

5 Conclusion

We proposed a method to reduce the computational cost of existing convolutional neural networks, by evaluating images in blocks and adjusting the processing resolution of each block dynamically. We introduced custom block operations, enabling efficient inference and training of these dynamic neural networks. Our BlockPad module is essential to enable feature propagation between individual blocks. A lightweight policy network, trained with reinforcement learning, selects complex regions for high-resolution processing. Our method is demonstrated on semantic segmentation of street scenes and we show that the computational complexity of the SwiftNet architecture can be reduced with only a small decrease in accuracy. Dynamically adjusting the processing resolution per block achieves better accuracy than simply downscaling the input image to a lower resolution.

The idea of block-based processing and dynamic resolution adaptation can be integrated in various network architectures and computer vision tasks. In particular, our method is suited for dense pixel-wise classification tasks such as depth estimation, instance segmentation or human pose estimation. Furthermore, our block modules such as BlockPad pave the way towards other variants, for instance by processing blocks with varying quantization levels. Dynamic processing can help to keep the number of operations and energy consumption under control.

REFERENCES

[1] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The Cityscapes Dataset for Semantic Urban Scene Understanding," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 3213–3223.

[2] N. C. Codella, D. Gutman, M. E. Celebi, B. Helba, M. A. Marchetti, S. W. Dusza, A. Kalloo, K. Liopyris, N. Mishra, H. Kittler *et al.*, "Skin lesion analysis toward melanoma detection: A challenge at the 2017 international symposium on biomedical imaging (isbi), hosted by the international skin imaging collaboration (isic)," in *15th Int. Symp. on Biomedical Imaging (ISBI 2018)*. IEEE, 2018, pp. 168–172.

[3] I. Demir, K. Koperski, D. Lindenbaum, G. Pang, J. Huang, S. Basu, F. Hughes, D. Tuia, and R. Raskar, "DeepGlobe 2018: A Challenge to Parse the Earth through Satellite Images," in *IEEE Conf. on Computer Vision and Pattern Recognition Workshops (CVPRW)*. Salt Lake City, UT: IEEE, Jun. 2018, pp. 172–17 209.

[4] E. Maggiori, Y. Tarabalka, G. Charpiat, and P. Alliez, "Can semantic labeling methods generalize to any city? the inria aerial image labeling benchmark," in *IEEE Int. Geoscience and Remote Sensing Symp. (IGARSS)*, Jul. 2017, pp. 3226–3229.

[5] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4, pp. 834–848, Apr. 2018.

[6] A. Canziani, A. Paszke, and E. Culurciello, "An Analysis of Deep Neural Network Models for Practical Applications," *arXiv:1605.07678 [cs]*, Apr. 2017.

[7] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[8] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *IEEE Conf. on Computer Vision and Pattern Recognition*. Salt Lake City, UT: IEEE, Jun. 2018, pp. 4510–4520.

[9] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," in *Proc. European Conf. on Computer Vision (ECCV)*, 2018, pp. 116–131.

[10] H. Zhao, X. Qi, X. Shen, J. Shi, and J. Jia, "ICNet for Real-Time Semantic Segmentation on High-Resolution Images," in *Proc. European Conf. Computer Vision (ECCV)*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer Int. Publishing, 2018, vol. 11207, pp. 418–434.

[11] E. Romera, J. M. Álvarez, L. M. Bergasa, and R. Arroyo, "ERFNet: Efficient Residual Factorized ConvNet for Real-Time Semantic Segmentation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 263–272, Jan. 2018.

[12] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal Brain Damage," in *Advances in Neural Information Processing Systems 2*, D. S. Touretzky, Ed. Morgan-Kaufmann, 1990, pp. 598–605.

[13] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *arXiv:1503.02531 [cs, stat]*, Mar. 2015.

[14] J. Wu, C. Leng, Y. Wang, Q. Hu, and J. Cheng, "Quantized Convolutional Neural Networks for Mobile Devices," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 4820–4828.

[15] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning Dynamic Routing in Convolutional Networks," in *Proc. European Conf. on Computer Vision (ECCV)*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer Int. Publishing, 2018, vol. 11217, pp. 420–436.

[16] A. Veit and S. Belongie, "Convolutional networks with adaptive inference graphs," in *Proc. European Conf. on Computer Vision (ECCV)*, 2018, pp. 3–18.

[17] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "BlockDrop: Dynamic Inference Paths in Residual Networks," in *IEEE Conf. on Computer Vision and Pattern Recognition*. Salt Lake City, UT: IEEE, Jun. 2018, pp. 8817–8826.

[18] B. E. Bejnordi, T. Blankevoort, and M. Welling, "Batch-Shaping for Learning Conditional Channel Gated Networks," *arXiv:1907.06627 [cs, stat]*, Apr. 2020.

[19] T. Verelst and T. Tuytelaars, "Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Seattle, WA, USA: IEEE, Jun. 2020, pp. 2317–2326.

[20] Y.-H. Huang, M. Proesmans, S. Georgoulis, and L. Van Gool, "Uncertainty based model selection for fast semantic segmentation," in *16th Int. Conf. on Machine Vision Applications (MVA)*. Tokyo, Japan: IEEE, May 2019, pp. 1–6.

[21] T. Wu, Z. Lei, B. Lin, C. Li, Y. Qu, and Y. Xie, "Patch Proposal Network for Fast Semantic Segmentation of High-Resolution Images," *Proc. AAAI Conf. on Artificial Intelligence*, vol. 34, no. 07, pp. 12 402–12 409, Apr. 2020.

[22] Z. Wu, C. Shen, and A. v. d. Hengel, "Real-time Semantic Image Segmentation via Spatial Sparsity," *arXiv:1712.00213 [cs]*, Dec. 2017.

[23] X. Li, Z. Liu, P. Luo, C. C. Loy, and X. Tang, "Not All Pixels Are Equal: Difficulty-Aware Semantic Segmentation via Deep Layer Cascade," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Honolulu, HI: IEEE, Jul. 2017, pp. 6459–6468.

[24] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. Vetrov, and R. Salakhutdinov, "Spatially Adaptive Computation Time for Residual Networks," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Honolulu, HI: IEEE, Jul. 2017, pp. 1790–1799.

[25] D. t. Vooturi, G. Varma, and K. Kothapalli, "Dynamic Block Sparse Reparameterization of Convolutional Neural Networks," in *IEEE Int. Conf. on Computer Vision Workshop (ICCVW)*. Seoul, Korea (South): IEEE, Oct. 2019, pp. 3046–3053.

[26] M. Ren, A. Pokrovsky, B. Yang, and R. Urtasun, "SBNet: Sparse Blocks Network for Fast Inference," in *IEEE Conf. on Computer Vision and Pattern Recognition*. Salt Lake City, UT: IEEE, Jun. 2018, pp. 8711–8720.

[27] F. Sun, M. Qin, T. Zhang, L. Liu, Y.-K. Chen, and Y. Xie, "Computation on sparse neural networks: an inspiration for future hardware," *arXiv preprint arXiv:2004.11946*, 2020.

[28] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid Scene Parsing Network," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Honolulu, HI: IEEE, Jul. 2017, pp. 6230–6239.

[29] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.

[30] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," *arXiv preprint arXiv:1511.07122*, 2015.

[31] G. Neuhold, T. Ollmann, S. R. Bulo, and P. Kontschieder, "The Mapillary Vistas Dataset for Semantic Understanding of Street Scenes," in *IEEE Int. Conf. on Computer Vision (ICCV)*. Venice: IEEE, Oct. 2017, pp. 5000–5009.

[32] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, "BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation," in *Proc. European Conf. on Computer Vision (ECCV)*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer Int. Publishing, 2018, vol. 11217, pp. 334–349.

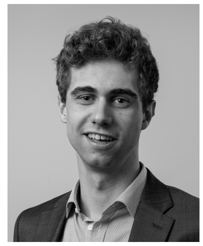
[33] G. Li, I. Yun, J. Kim, and J. Kim, "Dabnet: Depth-wise asymmetric bottleneck for real-time semantic segmentation," *arXiv preprint arXiv:1907.11357*, 2019.

[34] Z. Huang, X. Wang, L. Huang, C. Huang, Y. Wei, and W. Liu, "Ccnet: Criss-cross attention for semantic segmentation," in *Proc. IEEE Int. Conf. on Computer Vision*, 2019, pp. 603–612.

[35] F. Wu, F. Chen, X.-Y. Jing, C.-H. Hu, Q. Ge, and Y. Ji, "Dynamic attention network for semantic segmentation," *Neurocomputing*, vol. 384, pp. 182–191, Apr. 2020.

[36] Z. Zhu, M. Xu, S. Bai, T. Huang, and X. Bai, "Asymmetric Non-Local Neural Networks for Semantic Segmentation," in *IEEE Int. Conf. on Computer Vision (ICCV)*. Seoul, Korea (South): IEEE, Oct. 2019, pp. 593–602.

[37] P. Hu, F. Perazzi, F. C. Heilbron, O. Wang, Z. Lin, K. Saenko, and S. Sclaroff, "Real-time Semantic Segmentation with Fast Attention," *arXiv:2007.03815 [cs]*, Jul. 2020.
[38] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local Neural Networks," *arXiv:1711.07971 [cs]*, Apr. 2018.
[39] M. Orsic, I. Kreso, P. Bevandic, and S. Segvic, "In Defense of Pre-Trained ImageNet Architectures for Real-Time Semantic Segmentation of Road-Driving Images," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Long Beach, CA, USA: IEEE, Jun. 2019, pp. 12 599–12 608.
[40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 770–778.
[41] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *IEEE Conf. on Computer Vision and Pattern Recognition*, Jun. 2009, pp. 248–255.
[42] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 9, pp. 1904–1916, Sep. 2015.
[43] T. N. Sainath, B. Kingsbury, V. Sindhwani, E. Arisoy, and B. Ramabhadran, "Low-rank matrix factorization for Deep Neural Network training with high-dimensional output targets," in *IEEE Int. Conf. on Acoustics, Speech and Signal Processing*, May 2013, pp. 6655–6659.
[44] M. Jaderberg, A. Vedaldi, and A. Zisserman, "Speeding up Convolutional Neural Networks with Low Rank Expansions," *arXiv:1405.3866 [cs]*, May 2014.
[45] A. Recasens, P. Kellnhofer, S. Stent, W. Matusik, and A. Torralba, "Learning to zoom: a saliency-based sampling layer for neural networks," in *Proc. of the European Conf. on Computer Vision (ECCV)*, 2018, pp. 51–66.
[46] D. Marin, Z. He, P. Vajda, P. Chatterjee, S. Tsai, F. Yang, and Y. Boykov, "Efficient Segmentation: Learning Downsampling Near Semantic Boundaries," in *IEEE Int. Conf. on Computer Vision (ICCV)*. Seoul, Korea (South): IEEE, Oct. 2019, pp. 2131–2141.
[47] E. Bengio, P.-L. Bacon, J. Pineau, and D. Precup, "Conditional Computation in Neural Networks for faster models," *arXiv:1511.06297 [cs]*, Jan. 2016.
[48] Y. Bengio, N. Léonard, and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation," *arXiv:1308.3432 [cs]*, Aug. 2013.
[49] Y. Bengio, "Deep Learning of Representations: Looking Forward," *arXiv:1305.0445 [cs]*, Jun. 2013.
[50] Z. Xie, Z. Zhang, X. Zhu, G. Huang, and S. Lin, "Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation," *arXiv:2003.08866 [cs]*, Sep. 2020.
[51] M. Figurnov, A. Ibraimova, D. P. Vetrov, and P. Kohli, "Perforatedcnns: Acceleration through elimination of redundant convolutions," in *Advances in Neural Information Processing Systems*, 2016, pp. 947–955.
[52] W. Hua, Y. Zhou, C. M. De Sa, Z. Zhang, and G. E. Suh, "Channel gating neural networks," in *Advances in Neural Information Processing Systems*, 2019, pp. 1886–1896.
[53] E. Jang, S. Gu, and B. Poole, "Categorical Reparameterization with Gumbel-Softmax," *arXiv:1611.01144 [cs, stat]*, Aug. 2017.
[54] C. J. Maddison, A. Mnih, and Y. W. Teh, "The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables," *arXiv:1611.00712 [cs, stat]*, Mar. 2017.
[55] R. S. Sutton and A. G. Barto, *Reinforcement learning: an introduction*, ser. Adaptive computation and machine learning. Cambridge, Mass: MIT Press, 1998.
[56] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, no. 3-4, pp. 229–256, 1992.
[57] J. Zhuang, J. Yang, L. Gu, and N. Dvornek, "Shelfnet for fast semantic segmentation," in *Proc. IEEE Int. Conf. on Computer Vision Workshops (ICCVW)*, 2019.
[58] D. Mazzini, "Guided upsampling network for real-time semantic segmentation," *arXiv preprint arXiv:1807.07466*, 2018.



Thomas Verelst is a PhD student at the PSI group of the Electrical Engineering department of KU Leuven. He received a MS degree in Electrical Engineering at the same university in 2018. His research focuses on computer vision using efficient neural networks for classification, pose estimation and segmentation tasks.

Tinne Tuytelaars is a full professor at the Electrotechnical department of KU Leuven. Her research focuses on image understanding, with a focus on representation learning, multimodal learning (images and text) and continual learning. In 2009, she received an ERC starting independent researcher grant, and in 2016, she received the Koenderink award. She has been one of the Program Chairs of the European Conference on Computer Vision 2014 and of IEEE/CVF Conference on Computer Vision and Pattern Recognition 2021, and one of the General Chairs of IEEE/CVF Conference on Computer Vision and Pattern Recognition 2016. She has been associate editor in chief of the IEEE Transactions on Pattern Analysis and Machine Intelligence and serves as area editor for the International Journal on Computer Vision.